

Examen CSB 2026 — Automatización de APIs REST

Objetivo del examen

El objetivo es demostrar que puedes diseñar, implementar y ejecutar pruebas automatizadas para una API REST. No se evalúa una herramienta concreta, sino las siguientes competencias:

Competencia	Qué demuestra
Diseño de casos	Happy path, negativos, bordes
Flujos encadenados	Auth → crear → leer → actualizar → eliminar
Aserciones	Status, headers, schema, datos dinámicos
Manejo de datos	Variables, IDs generados, no hardcodear
Estructura	Separación por capas/features, legibilidad
Ejecución	CLI, reportes, reproducibilidad
Documentación	README con cómo correr y qué cubre

Herramientas permitidas (elige una)

- Karate DSL
- RestAssured (Java)
- Pytest + requests (Python)
- Playwright API / Supertest (Node)
- Cypress (API testing)

Prohibido

- Entregar solo requests manuales en Postman/Insomnia sin código versionado
- Usar Postman Collections como único entregable

Contexto

Automatiza la suite de pruebas para la API pública Restful Booker (<https://restful-booker.herokuapp.com/apidoc/index.html>).

API bajo prueba

#	Método	Endpoint	Auth	Descripción
1	POST	/auth	No	Obtener token con admin / password123
2	GET	/booking	No	Listar todos los bookings
3	GET	/booking/{id}	No	Obtener booking por ID
4	POST	/booking	No	Crear booking

#	Método	Endpoint	Auth	Descripción
5	PUT	/booking/{id}	Cookie token	Actualizar booking
6	DELETE	/booking/{id}	Cookie token	Eliminar booking

Base URL: <https://restful-booker.herokuapp.com>

Entregables obligatorios

- Repositorio Git personal** con todo el código de automatización. Debes:
 - Crear el repositorio en una cuenta personal de GitHub, GitLab o Bitbucket.
 - Subir todos los archivos necesarios para ejecutar el proyecto.
 - Compartir el enlace del repositorio al finalizar el examen.
 - Mantenerlo público durante la evaluación o dar acceso al evaluador si es privado.
 - Incluir varios commits con mensajes claros que permitan ver la evolución del trabajo.
- README.md** con:
 - Prerrequisitos (Java/Node/Python según stack)
 - Comando para ejecutar todas las pruebas
 - Comando o ubicación del reporte
- Mínimo 12 casos automatizados** según la rúbrica
- Al menos 1 flujo E2E** que encadene: auth → create → get → update → delete
- Archivo de configuración** para base URL (no hardcodear en cada test)

Instrucciones para la entrega

6. Crea un repositorio en tu cuenta personal de GitHub, GitLab o Bitbucket.
7. Sube tu solución completa, incluyendo el código, la configuración y el `README.md`.
8. Antes de entregar, clona el repositorio en otra carpeta y confirma que las pruebas se pueden ejecutar siguiendo únicamente las instrucciones del README.
9. Comparte con el evaluador la URL del repositorio y, si aplica, la ubicación del reporte generado.
10. No subas carpetas generadas, dependencias ni archivos del IDE, como `target/`, `node_modules/` o `.idea/`.

Casos obligatorios (12 mínimo)

Bloque A — Autenticación (2 casos)

ID	Caso	Resultado esperado
AUTH-01	Login con credenciales válidas	Status 200, token existe y es string no vacío
AUTH-02	Login con password incorrecto	Status 200 sin token; documentar body real (p.ej. { "reason": "Bad credentials" })

Bloque B — Lectura (3 casos)

ID	Caso	Resultado esperado
GET-01	GET /booking	Status 200, body es array con al menos 1 elemento
GET-02	GET /booking/{id} con un ID existente (obtenerlo de la lista o de un POST previo; no asumir que existe el 1)	Status 200, campos <code>firstname</code> , <code>lastname</code> , <code>totalprice</code> , <code>bookingdates</code> presentes
GET-03	GET /booking/99999	Status 404

Bloque C — Creación (2 casos)

ID	Caso	Resultado esperado
POST-01	Crear booking con payload válido	Status 200, retorna <code>bookingid</code> , datos coinciden con request
POST-02	Crear booking sin <code>firstname</code>	Status 400 o 500 (documentar y validar status real)

Bloque D — Actualización (2 casos)

ID	Caso	Resultado esperado
PUT-01	Actualizar booking existente con token válido	Status 200, <code>firstname</code> actualizado
PUT-02	Actualizar sin token / token inválido	Status 401, 403 o 405

Bloque E — Eliminación (2 casos)

ID	Caso	Resultado esperado
DEL-01	Eliminar booking con token válido	Status 201
DEL-02	Eliminar booking inexistente	Status 404 o 405 (documentar)

Bloque F — Flujo E2E (1 caso obligatorio)

ID	Caso	Pasos
E2E-01	Ciclo de vida completo	Auth → POST booking → GET por id → PUT → GET verificar cambio → DELETE → GET verificar 404

Criterios de evaluación (100 puntos)

Criterio	Puntos
Casos funcionan y pasan contra la API real	30
Flujo E2E con manejo correcto de token e IDs dinámicos	20
Aserciones completas (status + body + tipos)	15
Estructura del proyecto (features/clases organizadas)	10
Casos negativos implementados	10
Configuración externalizada (baseUrl, credenciales)	5
README claro y reproducible	5
Código legible (nombres, sin duplicación excesiva)	5

Nota mínima para aprobar: 70/100

Restricciones

- No usar Postman Collections como entregable
- No commitear credenciales reales de producción (aquí son de demo, está bien)
- Los IDs de booking **no deben estar hardcodeados** en el flujo E2E (usar el ID retornado por POST)
- Configura headers `Accept` y `Content-Type: application/json`. La API a veces responde **418 I'm a Teapot** si faltan headers o hay rate limiting; investiga y documenta el comportamiento.

Bonus (+10 puntos opcionales)

- Validación de schema JSON (ej. `bookingdates.checkin` formato fecha)
- Ejecución en CI (GitHub Actions / Jenkinsfile mínimo)
- Data-driven: mismo test con 2 payloads distintos de creación

Checklist de autoevaluación (candidato)

- ☐ Subí el código a un repositorio de mi cuenta personal

- ☐ El evaluador puede acceder al repositorio
- ☐ El repositorio contiene varios commits con mensajes claros
- ☐ Puedo ejecutar todas las pruebas con un solo comando
- ☐ El E2E usa ID dinámico del POST
- ☐ El token se obtiene en runtime
- ☐ Hay al menos 2 casos negativos
- ☐ README permite correr sin preguntarle nada al autor
- ☐ Auth / CRUD / E2E están separados
- ☐ Las aserciones van más allá de solo `status 200`